Note

The objective of any RL algorithm is to maximize the rewards over the course of an entire episode.

Since our original *n*-armed bandit problem did not have a state space, only an action space, we just needed to learn the relationship between actions and rewards. We learned the relationship by using a lookup table to store our experience of receiving rewards for particular actions. We stored action-reward pairs (a_k, r_k) where the reward at play k was an average over all past plays associated with taking action a_k.

In our *n*-armed bandit problem, we only had 10 actions, so a lookup table of 10 rows was very reasonable. But when we introduce a state space with contextual bandits, we start to get a combinatorial explosion of possible state-action-reward tuples. For example, if we have a state space of 100 states, and each state is associated with 10 actions, we have 1,000 different pieces of data we need to store and recompute. In most of the problems we'll consider in this book, the state space is intractably large, so a simple lookup table is not feasible.

That's where deep learning comes in. When they're properly trained, neural networks are great at learning abstractions that get rid of the details of little value. They can learn composable patterns and regularities in data such that they can effectively compress a large amount of data while retaining the important information. Hence, neural networks can be used to learn complex relationships between state-action pairs and rewards without us having to store all such experiences as raw memories. We often call the part of an RL algorithm that makes the decisions based on some information the *agent*. In order to solve the contextual bandit we've been discussing, we'll employ a neural network as our agent.

First, though, we will take a moment to introduce PyTorch, the deep learning framework we will be using throughout this book to build neural networks.